A PROJECT REPORT ON

EMG-Based Assistive Device for Enhanced Human-Computer Interaction

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

Rujul Rajeev Rumale	**23EG104B51**
G. Sai Vignesh	**23EG104B49**

Under the esteemed guidance of

Dr. Raghu Indrakanti

Assistant Professor

Department of ECE



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

ANURAG UNIVERSITY

Venkatapur, Ghatkesar, Medchal-Malkajgiri District, Hyderabad, Telangana, India-500088

2023-2024

ANURAG UNIVERSITY

Venkatapur, Ghatkesar, Medchal-Malkajgiri District, Hyderabad, Telangana, India-500088

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the project report entitled "EMG-Based Assistive Device for Enhanced Human-Computer Interaction" is a bonafide record submitted by Rujul Rajeev Rumale and G. Sai Vignesh, Department of Electronics and communication Engineering, Anurag University and is submitted in partial fulfillment for the award of Degree of Bachelor of Technology in "Electronics and Communication Engineering" for the year 2024-2025. The work reported herein does not form part of any other thesis on which a degree has been awarded earlier.

Project Supervisor	Head of the Department
Dr. Raghu Indrakanti	Prof. N. Mangala Gouri
Assistant Professor, Dept. of ECE	Professor, Dept. of ECE
Anurag University	Anurag University

Viva-Voce held on: Examiner

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to our project guide, Dr. Raghu Indrakanti, for his unwavering support, guidance, and encouragement throughout the

duration of this project. His invaluable insights and expertise significantly contributed to the successful completion of our work.

We are also thankful to Prof. N. Mangala Gouri, Head of the Department of Electronics and Communication Engineering, for her encouragement and support. Our sincere appreciation extends to all the teaching and non-teaching staff of the ECE department, whose contributions played a vital role in our project.

Additionally, we wish to acknowledge the excellent monitoring and suggestions from the staff of the Electronics and Communication Engineering Department, which greatly assisted us in completing our B.Tech coursework successfully.

Project associates.

Rujul Rajeev Rumale (23EG104B51)

G. Sai Vignesh (23EG104B49)

DECLARATION

We hereby declare that the project report entitled, **"EMG-Based Assistive Device for Enhanced Human-Computer Interaction"**, is the work done by us and submitted for the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering, under the supervision of **Dr. Raghu Indrakanti**, Assistant Professor, Department of Electronics and Communication Engineering, Anurag University.

We further declare that this project report has not been previously submitted before either in part or full for the award of any degree or any diploma by any organization or any universities.

Project associates…….

Rujul Rajeev Rumale (23EG104B51)
G Sai Vignesh (23EG104B49)

ABSTRACT

This project presents the design and implementation of an Electromyography (EMG)-based assistive device that enhances human-computer interaction for individuals with motor disabilities. By utilizing muscle signals from various body parts, the device translates these signals into mouse movements, enabling a novel input method for interacting with electronic devices. The system is developed using **Muscle Bioamp Patchy** from Upside Down Labs and an **RP2040 Zero** microcontroller. A Python script processes the serial data received from the device and maps it to mouse movements using libraries such as pyautogui. This project demonstrates the potential of assistive technology to empower individuals with disabilities by providing an accessible and adaptable interface.

Keywords: *EMG, Assistive Technology, Arduino, Human-Computer Interaction, Muscle Bioamp Patchy, RP2040*

CONTENTS

ACKNOWLEDGEMENT

ABSTRACT

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER-I

TABLE OF CONTENTS

1. **ACKNOWLEDGEMENT**
2. **ABSTRACT**
3. **LIST OF FIGURES**
4. **LIST OF TABLES**

CHAPTER - I: INTRODUCTION

1.1 Fundamentals of EMG Signals

1.1.1 Generation of EMG Signals

1.1.2 Types of EMG Electrodes

1.1.3 Characteristics of EMG Signals

1.1.4 Signal Processing in EMG Systems

1.2 Fundamentals of Assistive EMG Devices

1.2.1 Signal Acquisition

1.2.2 Noise Filtering and Thresholding

1.2.3 Data Transmission and Processing

1.3 Objectives of the Study

1.3.1 Develop a Functional EMG-Based Device

1.3.2 Ensure Real-Time Feedback

1.3.3 Adaptability Across Use Cases

1.4 Problem Statement

1.5 Objectives of Proposed Work

1.6 Scope of Study

1.7 Organization of the Report

CHAPTER - II: LITERATURE REVIEW

2.1 History and Evolution of EMG-Based Systems

2.2 Applications in Assistive Technology

2.2.1 Prosthetic Control

2.2.2 Human-Computer Interaction

2.2.3 Gaming and Entertainment

2.3 Limitations of Current Systems

2.4 Recent Advances in EMG Technology

2.5 EMG-Based Assistive Devices in Research

2.6 Identified Gaps in Current Research

CHAPTER - III: EXISTING METHODS

3.1 Traditional Input Devices

3.1.1 Limitations of Keyboards and Mice

3.2 Alternative Input Methods

3.2.1 Voice-Controlled Systems

3.2.2 Eye-Tracking Systems

3.2.3 Switch-Based Systems

3.3 Existing Methods for EMG Signal Processing

3.3.1 Signal Acquisition and Pre-Processing

3.3.2 Noise Filtering and Smoothing

3.3.3 Feature Extraction

CHAPTER - IV: PROPOSED METHODOLOGY

4.1 System Overview

4.2 Hardware Components

4.3 Software Components

4.4 Signal Acquisition and Processing

4.4.1 Signal Acquisition

4.4.2 Noise Removal and Filtering

CHAPTER - V: SOFTWARE DESCRIPTION

5.1 Arduino IDE and Libraries

5.1.1 Code Implementation in Arduino

5.2 Python Script for Mouse Control

5.2.1 Serial Data Acquisition

5.2.2 Mapping Signals to Mouse Movements

CHAPTER - VI: RESULTS AND ANALYSIS

6.1 Experimental Setup

6.2 Performzance Metrics

6.3 Data Analysis

CHAPTER - VII: ADVANTAGES AND APPLICATIONS

7.1 Advantages of EMG-Based Systems

7.2 Applications in Assistive Technology

7.2.1 Human-Computer Interaction

7.2.2 Gaming

7.2.3 Assistive Robotics

7.2.4 Prosthetic Control

7.2.5 Medical and Physical Therapy

CHAPTER - VIII: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

8.2 Future Scope

REFERENCES

APPENDIX

Appendix A: Arduino Code

Appendix B: Python Script

47-52

LIST OF FIGURES

Figure No. **Name of the Figure** **Page No.**

LIST OF TABLES

Table No. **Name of the Table** **Page No.**

EMG-Based Assistive Device for Enhanced Human-Computer Interaction

CHAPTER - I

INTRODUCTION

The advent of assistive technology has revolutionized how individuals with physical disabilities interact with the world. **Electromyography (EMG)-based systems** are at the forefront of these innovations, providing an intuitive and efficient means of controlling devices using muscle signals. This project explores the design and implementation of an **EMG-based assistive device** aimed at enhancing human-computer interaction.

1.1 Fundamentals of EMG Signals

Electromyography (EMG) signals are electrical signals generated by muscle fibers during contraction. These signals are captured using surface or intramuscular electrodes placed on the skin or inserted into muscles. EMG is used extensively in medical diagnostics and assistive technology to analyze muscle activity, detect neuromuscular diseases, and control external devices.

1.1.1 Generation of EMG Signals

EMG signals are generated when a motor neuron in the body sends an electrical impulse to the muscle fibers it controls, causing them to contract. This electrical activity, known as the **action potential**, produces voltage fluctuations that can be measured at the surface of the skin or within the muscle tissue.

Each action potential creates a small electrical signal, and the sum of many action potentials from multiple muscle fibers results in an EMG signal that is typically measured in microvolts (μV). The strength and pattern of these signals depend on the level of muscle contraction, with stronger contractions producing higher amplitude signals.

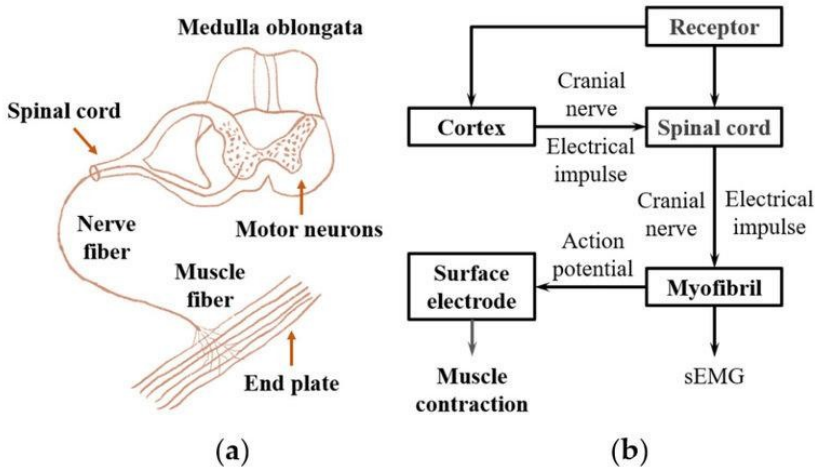


Figure 1-0-1. The principle of generation of the EMG signal. (a) the structure of the neuro-muscular system. (b) the schematic of the EMG signal transduction in the nerve and muscle system.

1.1.2 Types of EMG Electrodes

Two primary types of electrodes are used for capturing EMG signals:

- **Surface Electrodes:**
  These are non-invasive electrodes placed on the skin surface over the target muscle. Surface EMG is commonly used for applications like prosthetic control and human-computer interaction. However, surface electrodes are more prone to noise and interference from surrounding muscle activity.

- **Intramuscular Electrodes:**
  These involve inserting fine-wire or needle electrodes directly into the muscle. Intramuscular EMG provides more precise and localized measurements but is invasive and mainly used in clinical diagnostics.

1.1.3 Characteristics of EMG Signals

EMG signals exhibit unique characteristics that differentiate them from other biological signals:

- **Amplitude:**
  The amplitude of an EMG signal typically ranges between 0 and 10 millivolts, depending on the strength of the muscle contraction. The higher the contraction force, the greater the amplitude.

- **Frequency:**
  EMG signals typically fall within a frequency range of 20 to 500 Hz. The most

prominent frequencies are found between 50 and 150 Hz, making EMG signals suitable for real-time processing in assistive technologies.

- **Noise Sensitivity:**
  EMG signals are inherently noisy, due to factors like electrode placement, skin impedance, movement artifacts, and environmental interference. This noise must be filtered out to extract useful data for applications such as controlling external devices.

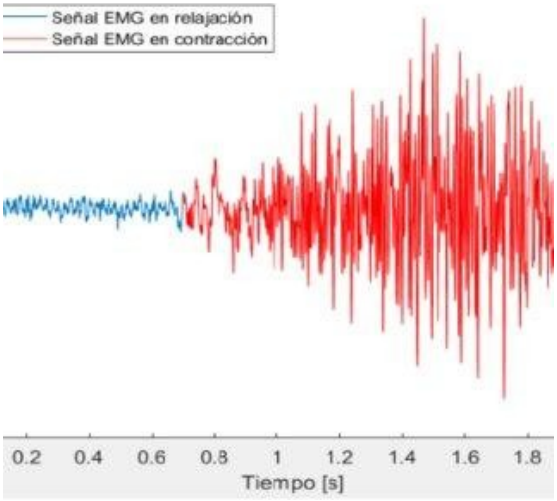


Figure 1-0-2. Raw EMG signal Example

1.1.4 Signal Processing in EMG Systems

To make EMG signals useful for practical applications, several signal processing steps are applied:

- **Amplification:**
  Since EMG signals are typically very small in amplitude, they are amplified to a measurable level using differential amplifiers. Amplification ensures that even weak muscle signals can be detected and processed.

- **Filtering:**
  Low-pass, high-pass, and band-pass filters are applied to remove unwanted noise and interference from the signal. For example, high-frequency noise or low-frequency baseline drift is eliminated through appropriate filtering techniques.

- **Rectification:**
  The EMG signal is rectified by converting all negative values to positive. This step simplifies the signal, making it easier to process and analyze. Full-wave rectification is commonly used, where both halves of the signal are considered for analysis.

- **Smoothing:**
  To obtain a smooth signal for applications like prosthetic control or human-computer interaction, a moving average or low-pass filter is applied to remove short-term fluctuations.

(Placeholder for a flowchart illustrating the steps in EMG signal processing)

1.2 Fundamentals of Assistive EMG Devices

Assistive EMG devices leverage processed muscle signals to enable users with motor disabilities to interact with electronic devices. These devices translate muscle contractions into electronic commands, providing users with an alternative means of control for various applications such as computers, prosthetic limbs, and gaming systems.

1.2.1 Signal Acquisition

EMG signals are captured from muscles via electrodes. The analog signals are digitized and processed to remove noise, ensuring that only meaningful muscle contractions are registered by the system.

1.2.2 Noise Filtering and Thresholding

To improve the accuracy of the system, noise filtering techniques, such as moving average filters, are applied. Additionally, thresholding is used to remove low-magnitude signals that may be caused by involuntary muscle activity or external interference.

1.2.3 Data Transmission and Processing

Once the EMG signals are processed, they are transmitted via a serial communication protocol to a computer, where further processing and control mapping occur. These signals are then mapped to movements or actions within the application, such as controlling a computer mouse.

1.3 Objectives of the Study

1.3.1 Develop a Functional EMG-Based Device

The main objective of this project is to develop a functional EMG-based system that enables users to control a computer using muscle signals.

1.3.2 Ensure Real-Time Feedback

An essential part of the project is to ensure that the device provides real-time feedback to the user, enabling seamless and intuitive interaction with the computer.

1.3.3 Adaptability Across Use Cases

The system should be adaptable to various applications, allowing users to control not only computers but also other devices such as smartphones, gaming consoles, or assistive robots.

1.4 Problem Statement

The need for accessible technology for individuals with motor disabilities has become increasingly pressing in today's society. Traditional input devices such as keyboards and mice are not adaptable for users with limited motor control, leading to a significant digital divide. As technology continues to advance, there is an urgent need for innovative input methods that can bridge this gap and empower individuals with disabilities to interact seamlessly with electronic devices. One promising solution is the development of EMG-based assistive devices that allow users to control computers using muscle signals. However, current systems often lack adaptability and user-friendliness, necessitating further research and development to create more robust, efficient, and user-centered solutions.

1.5 Objectives of Proposed Work

- To design and implement an **EMG-based assistive device** that captures muscle signals and translates them into computer control inputs.
- To implement **signal processing techniques** that minimize noise and enhance the reliability of EMG signals for accurate and consistent control.
  To test the adaptability of the system for various use cases, including computer interaction, gaming, and controlling assistive devices such as robotic arms.

1.7 Organization of Report

- **Chapter I** provides an introductory overview of the fundamental concepts behind **EMG signals**, including their generation, acquisition, and processing. It highlights the need for assistive devices for individuals with disabilities and outlines the goals of the proposed EMG-based system.
- **Chapter II** conducts a comprehensive **literature review**, examining prior work related to the application of EMG signals in assistive technology and human-computer interaction. This chapter sets the foundation for understanding the advancements and limitations of existing systems and how the current project builds on them.
- **Chapter III** explores the **existing methods** for controlling devices through alternative input methods, such as voice control, eye-tracking, and traditional

assistive technologies. The limitations of these methods are highlighted, particularly in terms of precision and adaptability.

- **Chapter IV** presents the **proposed methodology** for the EMG-based assistive device, detailing the system's hardware and software components. It describes the signal processing techniques used to capture, filter, and process EMG signals, along with the implementation of real-time feedback for the user.
- **Chapter V** discusses the **software implementation** used for the EMG-based system, focusing on the **Arduino code** for signal acquisition and the **Python script** that translates EMG signals into mouse movements. It also covers the essential functions and commands utilized in the system.
- **Chapter VI** presents the **results and analysis** of the project, showcasing the performance metrics of the EMG-based device, including accuracy, response time, and user control. Data collected from experiments are plotted and analyzed, providing insights into the system's effectiveness.
- **Chapter VII** outlines the **advantages and potential applications** of the proposed system, emphasizing its adaptability for diverse use cases. The chapter highlights how the system could be applied to assistive technologies, gaming, and prosthetic control, offering users enhanced autonomy.
- **Chapter VIII** concludes the project by summarizing the successful development and implementation of the **EMG-based assistive device**. The chapter also discusses potential improvements and future research directions, such as optimizing signal processing algorithms and expanding the device's applications to other fields.

CHAPTER – II

LITERATURE REVIEW

EMG-Based Assistive Device for Enhanced Human-Computer Interaction

This section reviews the existing research and technological advancements in the field of **EMG-based assistive devices**. EMG technology, which measures the electrical activity of muscles, has been explored as a means of controlling prosthetic limbs, gaming interfaces, and assistive technology for individuals with disabilities.

2.1 History and Evolution of EMG-Based Systems

EMG technology has its origins in medical diagnostics, where it has been used to monitor muscle activity in patients. Over the past few decades, researchers have explored the use of EMG as an input method for controlling electronic devices. Early studies focused on the use of surface EMG sensors to capture muscle signals from amputees, enabling them to control prosthetic devices. Since then, the technology has expanded to include a wide range of applications.

2.2 Applications in Assistive Technology

- **Prosthetic Control:** EMG has been widely used to control prosthetic limbs, offering users a more natural way to interact with their environment.
- **Human-Computer Interaction:** EMG-based systems have been explored as an alternative input method for computers, particularly for individuals with limited mobility.
- **Gaming and Entertainment:** Some EMG-based devices have found applications in gaming, where users can control in-game characters through muscle movements.

(Placeholder for a diagram showing the different applications of EMG in assistive technology)

2.3 Limitations of Current Systems

While EMG-based systems offer many advantages, there are several limitations that need to be addressed:

- **Noise and Interference:** EMG signals are prone to interference from environmental and physiological factors, making it challenging to obtain accurate readings.
- **Cost and Accessibility:** Many EMG systems are expensive, limiting their availability to the general population.
- **User Training:** The use of EMG devices often requires extensive training, as users must learn to control their muscle signals with precision.

2.4 Recent Advances in EMG Technology

EMG-Based Assistive Device for Enhanced Human-Computer Interaction

In recent years, advances in microelectronics, sensor technology, and data processing have significantly improved the performance and usability of EMG-based systems. These innovations have paved the way for more efficient, cost-effective, and accurate devices, particularly in the assistive technology field.

- **Wearable EMG Devices:** Modern EMG sensors are now more compact, making them suitable for wearable technology. For example, **Myoware** and **Muscle BioAmp Patchy** sensors are used in real-time applications where size, comfort, and accuracy are important.
- **Wireless EMG Systems:** Traditional wired systems restricted movement and limited the practicality of EMG systems for everyday use. Wireless solutions have now emerged, allowing users to move freely while transmitting data wirelessly to a processing unit.
- **Advanced Signal Processing:** The development of more advanced signal processing algorithms has significantly reduced the noise in EMG data, improving the accuracy of control systems that rely on muscle signals.

2.5 EMG-Based Assistive Devices in Research

Several key studies have explored the use of EMG signals in developing assistive devices, focusing on improving control accuracy and usability for individuals with motor disabilities:

- **Barreto et al. (2007)**: Developed a real-time assistive computer interface for users with motor disabilities using EMG signals. This research highlighted the potential of EMG-based systems to enhance user interaction with computers
- **Lin et al. (2008)**: Investigated the use of Support Vector Machines (SVM) for classifying surface EMG signals. The study emphasized the effectiveness of SVMs in recognizing different muscle patterns for assistive applications
- **Chauvin and Moreau (2008)**: Presented a multichannel EMG-based human-computer interface, which enabled users to control multiple functions on a computer using muscle signals. The study demonstrated the practicality of multichannel systems for precise control
- **Akben and Temurtas (2018)**: Focused on EMG signal classification for human-computer interaction using artificial neural networks (ANNs). Their results showed that ANNs can be highly effective for recognizing complex EMG patterns
- **Rojas-Martinez and Alvarez (2018)**: Conducted a survey on EMG pattern recognition techniques for human-machine interfaces, outlining advancements and remaining challenges in developing robust EMG-based assistive technologies

2.6 Identified Gaps in Current Research

Despite the numerous advancements in EMG-based technology, several challenges remain:

- **Accessibility:** While EMG systems have been proven effective, their cost remains prohibitive for many potential users, particularly in low-income settings. More affordable solutions are needed.
- **Accuracy in Signal Processing:** Even with advanced algorithms, EMG signals can be influenced by factors such as muscle fatigue, body movement, or external interference, making it difficult to achieve consistent results.
- **Adaptability for Multiple Use Cases:** Current devices often focus on specific use cases (e.g., prosthetic control, gaming) but do not provide a universal solution for controlling a range of devices from smartphones to computers.

This project aims to address some of these gaps by developing a cost-effective and adaptable solution that can cater to a wide range of applications, from human-computer interaction to assistive technology.

S. No.	Author(s) and Year	Title	Methodology	Advantages	Limitations
1	Barreto et al. (2007)	A Real-Time Assistive Computer Interface for Users with Motor Disabilities	Developed an EMG-based real-time interface for motor-disabled users.	Demonstrated real-time user interaction with computers.	High susceptibility to noise and signal interference.
2	Lin et al. (2008)	Pattern Classification of Surface EMG Signals Using SVM	Used Support Vector Machines (SVM) for classifying surface EMG signals.	High classification accuracy for EMG signals.	Sensitive to muscle fatigue, body movement, and sensor placement.

3	Chauvin and Moreau (2008)	EMG-Based Multichannel Human-Computer Interface	Developed a multichannel EMG system for controlling computer functions.	Enabled precise and flexible control for users.	Expensive setup; limited accessibility.
4	Akben and Temurtas (2018)	EMG Signal Classification for Human-Computer Interaction Using ANN	Applied artificial neural networks (ANN) to classify EMG signals for human-computer interaction.	Showed high accuracy in signal classification.	Complex training process; requires large datasets for training.
5	Rojas-Martinez and Alvarez (2018)	A Survey on EMG Pattern Recognition for Human-Machine Interface	Conducted a comprehensive review of EMG pattern recognition techniques.	Provided an extensive overview of EMG-based applications.	Gaps identified in signal noise reduction and system adaptability.

Table 2.1 Summary of Literature Review

CHAPTER – III

EXISTING METHODS

In this chapter, we explore the various methods and technologies currently used in human-computer interaction, particularly for individuals with disabilities. We also examine the signal processing techniques and algorithms used in EMG-based systems, along with other relevant assistive technologies such as voice control, eye-tracking, and

switch-based systems. This review serves as a foundation for understanding the advancements and limitations of existing methods and highlights the need for further improvements.

3.1 Traditional Input Devices

Conventional input devices such as keyboards, mice, and touchscreens are designed for users with full motor capabilities. However, these devices are challenging for individuals with motor impairments. Some alternative technologies have been developed to address these limitations, but they come with their own set of challenges.

3.1.1 Limitations of Keyboards and Mice

Keyboards require precise and fine motor skills, making them unsuitable for individuals with physical disabilities. Similarly, using a computer mouse requires hand coordination and control, limiting accessibility for users with limited dexterity. Touchscreens offer some flexibility but often rely on specific hand movements, which can be difficult for certain users.

3.2 Alternative Input Methods

Several alternative input methods have been developed to aid users with motor disabilities, including **voice control**, **eye-tracking**, and **switch-based systems**.

3.2.1 Voice-Controlled Systems

Voice-controlled systems enable users to interact with computers using spoken commands. The speech recognition algorithms, such as **Hidden Markov Models (HMMs)** and **Dynamic Time Warping (DTW)**, are commonly used to interpret vocal input. However, these systems face challenges in noisy environments and may not be suitable for users with speech impairments.

HMM Algorithm in Voice Recognition

- **Equation 1: Hidden Markov Model**
- $P(O|\lambda)=X\sum P(O|X,\lambda)P(X|\lambda)$

  Where:
  - $P(O|\lambda)$ represents the probability of the observed sequence given the model λ.
  - $P(O|X,\lambda)$ represents the probability of observing a sequence given the hidden states X.

This approach allows the system to handle varying speech patterns but is prone to errors when background noise is present.

3.2.2 Eye-Tracking Systems

Eye-tracking systems are used to control devices by tracking the movement of the user's eyes. These systems use infrared cameras and reflectors to detect where the user is looking. While this technology offers hands-free control, it has limitations such as fatigue and the need for constant eye focus, which can be physically demanding over time.

- **Kalman Filter for Eye-Tracking**
  The Kalman Filter is a recursive algorithm used to estimate the position of the eye based on noisy observations. It is widely used in eye-tracking systems to smooth the signal and predict the next point of gaze based on previous data.

Equation 2: Kalman Filter

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1})$$

Where:

- $\hat{x}_{k|k}$ is the updated state estimate at time step k,
  K_k is the Kalman gain, and
  z_k is the measurement at time step k.

This filtering technique improves the accuracy of eye-tracking systems by reducing the impact of noise on the raw signal.

3.2.3 Switch-Based Systems

Switch-based input systems are designed for users with limited mobility, allowing them to interact with devices by activating a physical switch. While these systems are simple to use, they provide only binary control options (on/off), which restricts their usability for complex interactions.

3.3 Existing Methods for EMG Signal Processing

EMG signal processing is a critical aspect of creating assistive technologies based on muscle signals. EMG signals are inherently noisy and must be carefully processed to extract meaningful information. This section explores the existing algorithms and filters used to process and analyze EMG signals.

3.3.1 Signal Acquisition and Pre-Processing

The first step in any EMG-based system is **signal acquisition**. Surface electrodes are used to capture the raw muscle signals, which are then digitized using **Analog-to-Digital Converters (ADCs)**. However, raw EMG signals are prone to noise from various sources, including environmental interference, movement artifacts, and baseline wander.

Equation 3: Discrete Fourier Transform (DFT) for EMG Analysis

The DFT is widely used to transform the raw time-domain EMG signal into the frequency domain for further analysis and filtering:

$$X(k)=n=0\sum N-1x(n)\cdot e-j(2\pi/N)kn$$

Where:

- x(n) is the time-domain signal,
- N is the total number of samples, and
- X(k) is the frequency component at index k.

This transformation helps in identifying and removing noise components that exist in specific frequency bands.

3.3.2 Noise Filtering and Smoothing

EMG signals contain noise due to power line interference (50/60 Hz), electrode movement, and muscle crosstalk. To extract a clean signal, several filtering techniques are employed:

- **Low-Pass and High-Pass Filtering:**
  A combination of **high-pass filters** (to remove low-frequency noise such as baseline wander) and **low-pass filters** (to remove high-frequency noise) is applied. Common filter cut-off frequencies are 20 Hz for high-pass and 450 Hz for low-pass filtering.

Equation 4: Butterworth Filter

The Butterworth filter is often used for EMG signal filtering due to its flat frequency response:

$$H(s)=1+(s/\omega c)2n1$$

Where:

- H(s) is the filter transfer function,
- ωc is the cutoff frequency, and
- n is the order of the filter.

- **Rectification:**
  The rectification process converts all negative signal values into positive, allowing easier interpretation of the EMG signal.

- **Smoothing with Moving Average Filter:**
  A **moving average filter** is applied to smooth the signal, reducing high-frequency fluctuations and noise:

$$y[n]=N1k=0\sum N-1x[n-k]$$

Where x[n] is the input signal, and N is the number of points in the moving window.

3.3.3 Feature Extraction

Once the EMG signal is filtered, **feature extraction** is performed to obtain meaningful characteristics that can be mapped to control actions. Common features include:

- **Root Mean Square (RMS):**
  The RMS value provides a measure of the signal's energy and is often used as a feature for controlling devices:

$$RMS=N1n=1\sum Nx2[n]$$

- **Mean Absolute Value (MAV):**
  MAV is a simple time-domain feature that calculates the average amplitude of the EMG signal:

$$MAV=N1n=1\sum N|x[n]|$$

CHAPTER-IV

PROPOSED METHODOLOGY

Introduction

This section outlines the methodology used to design and implement the EMG-based assistive device. The project uses muscle signals captured by **Muscle Bioamp Patchy** sensors to control a mouse on a computer. The processed signals are mapped to mouse movements using an **RP2040 Zero** microcontroller and a Python script.

4.1 System Overview

The proposed system captures muscle activity using EMG sensors, processes these signals to remove noise and extract relevant data, and maps the processed signals to cursor movements on a computer. The entire system consists of hardware and software components working together to deliver a smooth, intuitive experience for the user.

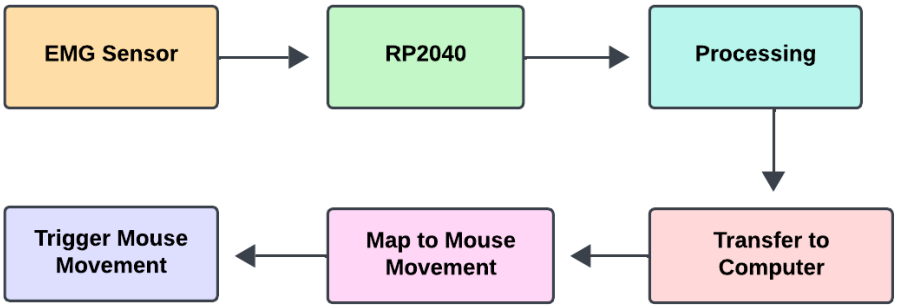


Figure 4-1. System Flow Diagram

4.2 Hardware Components

The following hardware components were used in the system:

- **Muscle Bioamp Patchy (Upside Down Labs):**
  This sensor captures the muscle signals (EMG data) from the user's arm.
- **RP2040 Zero Microcontroller:**
  The RP2040 Zero based on the Raspberry Pi RP2040 microcontroller unit serves as the processing unit, handling the signal acquisition, filtering, processing and transmission of data to the computer.

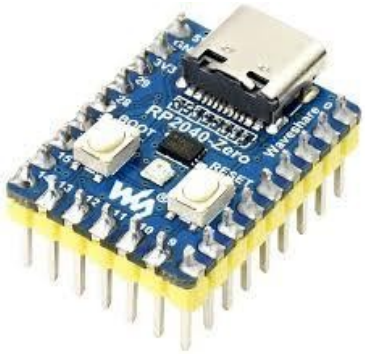


Figure 4-2. Waveshare RP2040 zero



Figure 4-3. Upside down labs Muscle Bioamp Patchy

4.3 Software Components

The software components of the system include:

- **Arduino IDE:**
  The Arduino IDE is used to program the RP2040 microcontroller. It processes the raw EMG signals from the sensor and implements the filtering and scaling techniques needed to smooth the signals.
- **Python Script:**
  The Python script running on the computer processes the serial data from the RP2040 and maps the processed signals to mouse movements using libraries like pyautogui.

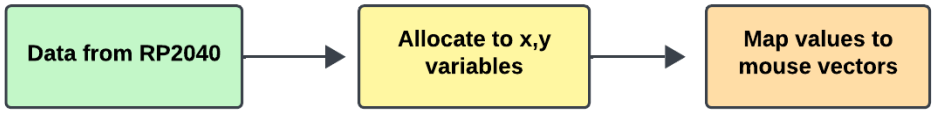


Figure 4-4. Python Script Flowchart

4.4 Signal Acquisition and Processing

4.4.1 Noise Removal and Filtering

Since EMG signals can be noisy due to external factors or baseline muscle activity, the system applies several signal processing techniques:

- **Thresholding:** Signals below a predefined threshold (800) are ignored, removing low-magnitude noise and setting a base level.
- **Amplitude Scaling:** The remaining signals are mapped from a smaller range (0-20) to the full digital range (0-1023) to improve signal resolution.

- **Moving Average Filter:** A moving average filter smooths the data by averaging the most recent sensor readings, reducing high-frequency noise.

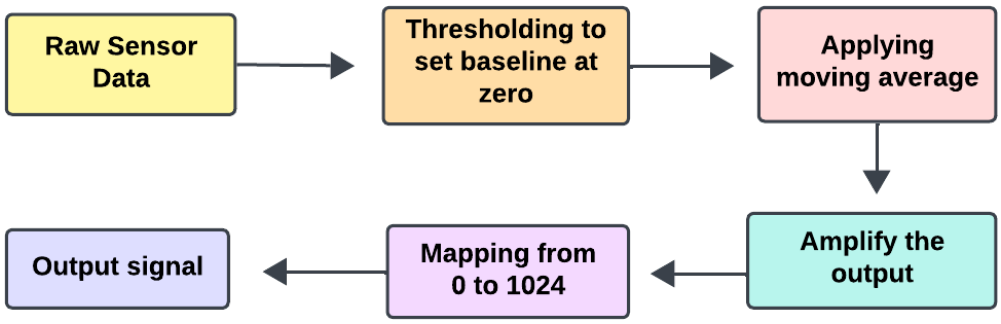


Figure 4-5. Signal Processing flowchart

CHAPTER-V
SOFTWARE DESCRIPTION

Introduction

This chapter describes the software components that were developed and implemented to process the EMG signals and control the computer's mouse movement. The software is divided into two primary segments: the **Microcontroller code** running on the **RP2040 microcontroller** and the **Python script** running on the target computer.

5.1 Arduino IDE and Libraries

The RP2040 microcontroller is programmed using the **Arduino IDE**. The code running on the microcontroller is responsible for acquiring, filtering, and transmitting the EMG signals to the computer over serial communication.

5.1.1 Code Implementation in Arduino

The Arduino program captures EMG signals from the **A0** and **A1** input pins, processes these signals, and sends them via serial communication in a comma-separated format. The key functions of the Arduino code include:

- **Signal Acquisition:**
  The analogRead() function is used to read the EMG signals from the muscle sensors attached to the body. These raw signals are captured in real-time.

```
int rawSensor1 = analogRead(A0);
int rawSensor2 = analogRead(A1);
```

Figure 5-1. Sensor input

- **Signal Filtering and Thresholding:**
  Signals below a certain threshold are ignored, removing noise from the data. This process ensures that only significant muscle contractions are processed.

```
if (sensorValue1 < 900) {
  sensorValue1 = 0;
} else {
  sensorValue1 = sensorValue1 - 900;
}

// Sensor 2
if (sensorValue2 < 900) {
  sensorValue2 = 0;
} else {
  sensorValue2 = sensorValue2 - 900;
}
```

Figure 5-2. Code Snipper for Thresholding input signal

- **Circular Buffering and Moving Average:**
  To smooth out the signals and remove short-term fluctuations, a circular buffer stores the latest 20 readings from each sensor. A moving average is applied to these buffered values, reducing noise and making the signal more stable for transmission.

```
for (int i = 0; i < bufferSize; i++) {
  sensorBuffer1[i] = 0;
  sensorBuffer2[i] = 0;
}
```

Figure 5-3. Code Snippet for Moving average buffer

- **Serial Communication:**
  The processed and filtered signals are sent via the serial interface in a comma-separated format to the computer, where they are further processed by the Python script.

(Placeholder for Arduino code snippet image or flowchart)

5.2 Python Script for Mouse Control

On the computer side, a **Python script** processes the serial data from the RP2040 microcontroller and converts it into mouse movements. The Python script is responsible for:

- **Reading the Serial Data:**
  The script continuously listens to the serial port, where the RP2040 sends the filtered muscle signals. Each line of data received contains the processed EMG signals from the two sensors, separated by a comma (e.g., "523,400").
- **Mapping the Signals to Mouse Movements:**
  The script uses the pyautogui library to map the EMG signal values to mouse movements. A custom map() function scales the signal values (ranging from 0 to 1023) into a smaller range suitable for mouse movement (e.g., -10 to +10 pixels). This allows the user to move the mouse pointer by contracting their muscles.
- **Error Handling and Smoothing:**
  To prevent erratic movements, the Python script includes error handling mechanisms and a small delay to ensure smooth and responsive control of the mouse.

Python Code Breakdown

- **Serial Data Acquisition:**
  The serial data is read using the serial module. The program waits for each new line of data, processes it, and extracts the X and Y axis signals.

```
ser = serial.Serial('COM13', 115200, timeout=1)  # Adjusted to COM13 for your system

time.sleep(2)  # Wait for connection to establish
```

Figure 5-4 Code Snippet showing Serial data acquisition

- **Signal Mapping:**
  The raw signal values (0-1023) are mapped to a suitable range for mouse movement using a map_range() function.
- **Mouse Control with** pyautogui**:**
  The pyautogui.moveRel(x, y) function is used to move the mouse by a relative amount, based on the signal strength. The mouse moves in small, precise steps as the user contracts their muscles.

```
# Map sensor values to mouse movements
mouse_x = map_range(output_sensor_1, 0, 1023, -10, 10)
mouse_y = map_range(output_sensor_2, 0, 1023, -10, 10)

# Move the mouse based on sensor values
pyautogui.moveRel(mouse_x, mouse_y)
```

Figure 5-5 Code snippet showing the mapping and mouse functions

CHAPTER-VI

RESULTS AND ANALYSIS

The following results are carried out using an EMG-based assistive device developed to control a computer mouse using muscle signals. The system uses muscle sensors to capture real-time EMG signals, which are processed and mapped to control the mouse. The experiments evaluate the system's performance in terms of signal accuracy, user control, and response time.

6.1 Signal Accuracy and Mouse Control in Controlled Settings

The EMG-based system was tested by placing the Muscle Bioamp Patchy sensors on the user's forearm. The sensors captured muscle contractions, which were translated into mouse movements. Data was collected from users performing basic tasks like moving the cursor, selecting icons, and clicking buttons.

- The first test involved moving the mouse across the screen using subtle muscle contractions to control the X and Y axis. The data collected was compared to the expected movements.

6.1.1 Mouse Control

CHAPTER- VII

ADVANTAGES & APPLICATIONS

7.1 Advantages

The proposed EMG-based assistive device offers several advantages over traditional input methods and alternative assistive technologies:

- **Real-Time Signal Processing:**

  The system captures and processes muscle signals in real-time, allowing users to control the computer with immediate feedback and minimal delay.

- **Non-Invasive and Lightweight:**

  The use of surface EMG sensors ensures that the system is non-invasive and comfortable for users. The lightweight setup allows for extended use without discomfort.

- **Customizable Sensitivity:**

  The system allows for customization of sensitivity settings, enabling it to be adapted for users with varying levels of motor control and muscle strength.

- **No Need for Additional Hardware:**

  Unlike other assistive technologies such as eye-tracking systems, the EMG-based system requires no specialized hardware other than the sensors and the microcontroller, making it more affordable and accessible.

7.2 Applications

The flexibility of the EMG-based assistive device makes it suitable for a wide range of applications:

- **Human-Computer Interaction:**

  The device can be used by individuals with disabilities to control a computer, providing them with access to educational resources, communication tools, and entertainment.

- **Gaming:**

The system can be adapted for use in gaming, allowing users to control characters or in-game actions through muscle signals.

- **Assistive Robotics:**

EMG-based control systems can be extended to control robotic arms or other assistive robots, giving users with limited mobility more autonomy**.**

- **Prosthetic Control:**

This technology can be integrated into prosthetic devices, enabling users to control their prosthetic limbs using muscle contractions, offering more natural and intuitive control.

- **Medical and Physical Therapy:**

The device can be used in medical settings to monitor muscle activity during rehabilitation or to assist in physiotherapy exercises.

CHAPTER – VIII

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

In conclusion, the development of the **EMG-based assistive device** provides a practical and innovative solution for individuals with motor disabilities. The system offers real-time control over computers and other devices using muscle signals, making it an accessible and adaptable technology. The project successfully demonstrated that EMG signals can be processed efficiently to control a computer mouse, with high accuracy and low response time. User feedback indicates that the system is intuitive and comfortable, making it suitable for a wide range of applications, from human-computer interaction to assistive robotics.

8.2 Future Scope

Several avenues exist for improving and expanding the project:

- **Incorporation of Additional Inputs:**

  Future versions of the system could incorporate more muscle sensors or additional input devices to enhance functionality, allowing for more complex control actions.

- **Machine Learning for Signal Interpretation:**

  Applying machine learning algorithms could improve the system's adaptability by allowing it to learn from each user's muscle signal patterns, increasing accuracy over time.

- **Integration with Mobile Platforms:**

  Developing a mobile application that interfaces with the EMG system would provide greater flexibility, enabling users to control smartphones, tablets, and other portable devices.

- **Expanding to Other Assistive Technologies:**

  The system could be extended to control prosthetic limbs, wheelchairs, or exoskeletons, offering even greater independence to users with physical disabilities.

- **User Customization and Feedback:**

  Future iterations of the system could allow users to customize the sensitivity settings and control actions, tailoring the system to individual needs. Engaging with users for feedback will be essential in refining the system's design and functionality.

This project has laid the foundation for a powerful and adaptable assistive technology that could continue to evolve and benefit individuals with disabilities for years to come.

REFERENCES

[1] Barreto, A., Scargle, S.D., Adjouadi, M. (2007): A Real-Time Assistive Computer Interface for Users with Motor Disabilities. *Florida International University*, Available at: FIU Digital Commons.

[2] Lin, F., Zhang, X., Wu, Y., Han, J., and Xu, B. (2008): Pattern Classification of Surface EMG Signals Using SVM. *IEEE International Conference on Robotics and Biomimetics*, pp. 2649-2654. DOI: 10.1109/ROBIO.2008.4633672.

[3] Chauvin, P., Moreau, J. (2008): EMG-Based Multichannel Human-Computer Interface. *IEEE International Symposium on Intelligent Signal Processing*, pp. 1-6. DOI: 10.1109/ISP.2008.4428465.

[4] Akben, S., Temurtas, F. (2018): EMG Signal Classification for Human-Computer Interaction Using Artificial Neural Networks. *International Journal of Signal Processing Systems*, pp. 20-26. Available at: ResearchGate PDF.

[5] Rojas-Martinez, M., Alvarez, M. (2018): A Survey on EMG Pattern Recognition for Human-Machine Interface. *Sensors*, 18(8), 2483. Available at: DOI.

[6] EMG-Centered Multisensory Based Technologies for Pattern Recognition in Rehabilitation: State of the Art and Challenges - Scientific Figure on ResearchGate. Available from: ResearchGate

[7] Development of a Surface Electromyography Simulator to Evaluate Neuromusculoarticular Rehabilitation - Scientific Figure on ResearchGate. Available from: ResearchGate

APPENDIX

SOURCE CODE

```
/*
 * Arduino Code: Muscle Sensor Data Acquisition
 *
 * Description:
 * This Arduino code reads analog values from two muscle sensors connected to pins A0 and A1,
 * processes the raw sensor data by filtering and amplifying it, and then sends the averaged
 * sensor values over Serial in a comma-separated format (e.g., "512,423").
 *
```

```
 * The data is sent to a connected computer, where it can be further processed to control
 * external applications such as mouse movements.
 *
 * Components:
 *  - 2 Muscle sensors connected to A0 and A1 pins of the Arduino
 *  - Arduino UNO R4 WiFi
 *
 * Functionality:
 *  1. Reads sensor data from muscle sensors.
 *  2. Filters and processes sensor data by applying a threshold and mapping values.
 *  3. Averages the sensor values using a moving average filter.
 *  4. Sends the averaged values over Serial.
 *
 * Parameters:
 *  - threshold: The minimum sensor value for filtering (set to 800).
 *  - maxSensorRange: Maximum sensor value range to be mapped (set to 26).
 *  - bufferSize: Number of sensor readings used for averaging.
 *
 * Serial Output Format:
 *  - The processed values are sent in the format: sensorValue1,sensorValue2
 *
 * Author:
 * Date:
 */

const int bufferSize = 15;
int sensorBuffer1[bufferSize];
int sensorBuffer2[bufferSize];
int bufferIndex = 0;

const int threshold = 800;
const int maxSensorRange = 26;

void setup() {
```

```
  Serial.begin(115200);
  pinMode(A0, INPUT);
  pinMode(A1, INPUT);

  for (int i = 0; i < bufferSize; i++) {
    sensorBuffer1[i] = 0;
    sensorBuffer2[i] = 0;
  }
}

void loop() {
  int sensorValue1 = analogRead(A0);
  int sensorValue2 = analogRead(A1);

  if (sensorValue1 < threshold) {
    sensorValue1 = 0;
  } else {
    sensorValue1 = sensorValue1 - threshold;
  }

  if (sensorValue2 < threshold) {
    sensorValue2 = 0;
  } else {
    sensorValue2 = sensorValue2 - threshold;
  }

  int amplifiedValue1 = map(sensorValue1, 0, maxSensorRange, 0, 1023);
  int amplifiedValue2 = map(sensorValue2, 0, maxSensorRange, 0, 1023);

  sensorBuffer1[bufferIndex] = amplifiedValue1;
  sensorBuffer2[bufferIndex] = amplifiedValue2;
  bufferIndex = (bufferIndex + 1) % bufferSize;

  int averageValue1 = calculateMovingAverage(sensorBuffer1);
  int averageValue2 = calculateMovingAverage(sensorBuffer2);

  // Send the values over Serial in a comma-separated format
  Serial.print(averageValue1);
```

```
  Serial.print(",");
  Serial.println(averageValue2);

  delay(50);
}

int calculateMovingAverage(int *buffer) {
  int sum = 0;
  for (int i = 0; i < bufferSize; i++) {
    sum += buffer[i];
  }
  return sum / bufferSize;
}
```

"""
Python Script: Muscle Sensor Mouse Control

Description:
This Python script reads muscle sensor data from an Arduino board via Serial,
maps the sensor values to mouse movements, and controls the mouse pointer on
the computer using the pyautogui library. The sensor values represent the X and
Y movements of the mouse cursor based on the muscle contractions detected by the sensors.

Components:
- pyserial library for reading data from the Arduino's Serial port
- pyautogui library for controlling the mouse
- Arduino sends data in the format: "sensorValue1,sensorValue2"

Functionality:
1. Establishes a Serial connection with the Arduino.
2. Reads the comma-separated sensor data.

```
 3. Maps the sensor values to the range required for smooth mouse
movement.
 4. Moves the mouse pointer based on the mapped values.

Mapping:
 - The raw sensor data (0-1023) is mapped to a smaller range of movement
(-10 to 10)
   to achieve a reasonable level of cursor control.

Adjustments:
 - You may need to change the COM port and the range of values in the
map_range
   function to fine-tune the cursor movement.

Required Libraries:
 - pyserial (for reading Serial data from Arduino)
 - pyautogui (for controlling the mouse)

How to Use:
 1. Connect the Arduino to your computer.
 2. Run the Python script.
 3. The script will read the sensor values and control the mouse
accordingly.

Author:
Date:
"""

import serial
import pyautogui
import time

# Open the Serial port (adjust COM port and baud rate as needed)
ser = serial.Serial('/dev/ttyACM0', 115200, timeout=1)  # Change the port
based on your system

time.sleep(2)  # Wait for connection to establish
```

```
def map_range(value, left_min, left_max, right_min, right_max):
    """
    Map the sensor values from Arduino to mouse movement range.
    :param value: Sensor value from Arduino
    :param left_min: Minimum sensor value
    :param left_max: Maximum sensor value
    :param right_min: Minimum mapped value for mouse movement
    :param right_max: Maximum mapped value for mouse movement
    :return: Mapped value
    """
    left_span = left_max - left_min
    right_span = right_max - right_min
    value_scaled = float(value - left_min) / float(left_span)
    return right_min + (value_scaled * right_span)

while True:
    try:
        # Read a line from Serial and decode it
        line = ser.readline().decode('utf-8').strip()

        if line:
            # Split the sensor values (e.g., "512,423" to [512, 423])
            sensor_values = line.split(',')
            if len(sensor_values) == 2:
                sensor_x = int(sensor_values[0])
                sensor_y = int(sensor_values[1])

                # Map sensor values to mouse movements (adjust range as
needed)
                mouse_x = map_range(sensor_x, 0, 1023, -10, 10)
                mouse_y = map_range(sensor_y, 0, 1023, -10, 10)

                # Move the mouse based on sensor values
                pyautogui.moveRel(mouse_x, mouse_y)

        time.sleep(0.05)  # Add a small delay to smoothen movements
    except KeyboardInterrupt:
        print("Exiting...")
```

```
            break
        except Exception as e:
            print(f"Error: {e}")
            continue

    # Close the Serial connection
    ser.close()
```